AWS SAA-C03 Study Notes: IAM Advanced & Organizations

1. AWS Organizations - Overview

AWS Organizations helps you centrally manage and govern your environment as you grow and scale your AWS resources. It allows you to consolidate multiple AWS accounts into an organization that you create and centrally manage.

- **Key Concepts:**
  - **Management Account:** The account that creates the Organization. It pays the bills (Consolidated Billing) and has full administrative control. **Critical:** Service Control Policies (SCPs) do *not* apply to the Management Account.
  - **Member Accounts:** All other accounts in the organization.
  - **Organizational Units (OUs):** A container for accounts within a root. You can apply policies to an OU, which then apply to all accounts within that OU.
  - **Consolidated Billing:** Aggregates usage costs across all accounts. You get volume discounts (e.g., S3 storage tiers, EC2 data transfer) by combining usage.
- **Service Control Policies (SCPs):**
  - SCPs allow you to restrict which AWS services, resources, and API actions can be accessed by users and roles in member accounts.
  - SCPs constitute a **permission boundary** for the account. They do *not* grant permissions; they only filter them. A user needs both an IAM permission *and* an SCP allowance to perform an action.
  - **Inheritance:** Policies filter down from Root -> OU -> Member Account.
  - **Allow vs. Deny:** Explicit Deny in an SCP overrides any Allow in IAM.

> **Exam Tip:** If a user in a Member Account has full AdministratorAccess in IAM but the SCP denies `ec2:*`, the user cannot access EC2. SCPs are the ultimate guardrail.

2. Organizations - Tag Policies

Tag policies represent a type of policy that helps you standardize tags across resources in your organization's accounts.

- **Purpose:** Enforce consistent tagging schema (e.g., ensuring every resource has a `CostCenter` tag with specific allowed values).
- **Enforcement:**
  - You can define compliant tag keys and values.
  - You can prevent non-compliant tagging operations on supported resources.
- **Integration:** Works with AWS Resource Groups and Tag Editor to audit compliance.

> **Exam Tip:** Use Tag Policies for cost allocation and governance. If a question asks how to ensure all specific resources created in the org have a specific tag key/value, the answer is Tag Policies.

3. IAM - Advanced Policies

Beyond basic Allow/Deny, IAM supports advanced conditions and boundaries.

- **IAM Permission Boundaries:**
  - An advanced feature used to set the **maximum permissions** that an identity-based policy can grant to an IAM entity (User or Role).
  - Commonly used when delegating administration (e.g., letting a developer create roles but restricting those roles to only S3 access).
  - *Logic:* Effective Permissions = intersection of (Identity-based Policy) AND (Permissions Boundary).
- **Policy Structure & Global Condition Keys:**
  - `aws:SourceIp` : Restrict access to specific IP ranges.
  - `aws:RequestedRegion` : Restrict calls to specific AWS Regions.
  - `aws:PrincipalOrgID` : Limit resource access to principals belonging to your specific AWS Organization (prevents access from external accounts).
  - `aws:MultiFactorAuthPresent` : Enforce MFA for sensitive operations.

```
/* Example: Deny access outside of specific regions */
{
    "Effect": "Deny",
    "NotAction": [
        "cloudfront:*",
        "iam:*",
        "route53:*",
        "support:*"
    ],
    "Resource": "*",
    "Condition": {
        "StringNotEquals": {
            "aws:RequestedRegion": [
                "us-east-1",
                "us-west-1"
            ]
        }
    }
}
```

4. IAM - Resource-based Policies vs IAM Roles

There are two primary ways to grant cross-account access or complex permissions.

Resource-based Policies

- **Definition:** Policy attached directly to the resource (e.g., S3 Bucket Policy, SQS Queue Policy, SNS Topic Policy, Lambda Function Policy, KMS Key Policy).
- **Principal:** You specify the `Principal` (User, Role, or Account ID) in the policy itself.
- **Use Case:** Simple cross-account access for supported resources (S3, SQS, SNS).
- **Benefit:** The principal does not need to give up their current permissions to access the resource; they stay in their own account context.

IAM Roles (Identity-based)

- **Definition:** An identity with specific permissions that can be assumed by a trusted entity.
- **Mechanism:** The user assumes the role via `sts:AssumeRole`. This returns temporary security credentials.
- **Use Case:** Cross-account access for services *without* resource-based policies (e.g., DynamoDB, EC2), or when a user needs to "switch contexts" completely.
- **The "Confused Deputy" Problem:** When a third party uses their access to your resources to access other resources.
  - **Solution:** Use `sts:ExternalId` condition in the Trust Policy when allowing 3rd party vendors to assume roles in your account.

> **Exam Tip:**
>
> - S3 Cross-Account Access? -> S3 Bucket Policy (Resource-based) is often the simplest answer unless you need programmatic access to many services.
> - EC2 Cross-Account Access? -> Must use IAM Roles.

5. IAM - Policy Evaluation Logic

How does AWS decide to Allow or Deny a request?

1. **Implicit Deny:** By default, everything is denied.
2. **Explicit Deny:** The evaluator looks for any policy (Identity, Resource, SCP, Boundary) that says `"Effect": "Deny"`.
   - **RULE:** An Explicit Deny **always** overrides any Allow.
3. **Explicit Allow:** If there is no Explicit Deny, the evaluator looks for at least one `"Effect": "Allow"`.
4. **Final Decision:**
   - If Explicit Deny found -> **DENY**
   - If No Allow found -> **DENY** (Implicit Deny)
   - If Allow found + No Explicit Deny -> **ALLOW**

Evaluation Boundary intersection:

- Identity Policy AND Permissions Boundary
- Identity Policy AND SCP (if in an Organization)
- Identity Policy AND Resource Policy (Union logic generally applies here, except for cross-account where *both* sides need to allow it).

6. AWS IAM Identity Center (Successor to AWS SSO)

- **Purpose:** Centralized login for all AWS accounts and business applications (Office 365, Salesforce).
- **Identity Source:**
  - Can use the built-in identity store.
  - Can connect to external Identity Providers (IdP) via **SAML 2.0** (e.g., Okta, Ping, Azure AD).
  - Can connect to Active Directory (via AWS Directory Services).
- **Features:**
  - **Single Sign-On:** Users log in once to a portal and click tiles to access specific AWS accounts with specific permission sets.
  - **Multi-Account Permissions:** Assign Permission Sets to groups for specific accounts.

> **Exam Tip:** If a question asks about "centrally managing access to multiple AWS accounts" or "integrating existing corporate credentials (SAML 2.0) for console access," the answer is IAM Identity Center.

7. AWS Directory Services

Brings Microsoft Active Directory (AD) to AWS.

Type	Description	Key Use Cases
AWS Managed Microsoft AD	Actual AD running on AWS infrastructure. Two domain controllers in different AZs.	• Establish **Trust Relationships** with on-prem AD. • Requires complex AD features (Group Policy, Schema extensions). • Seamlessly join EC2 Windows instances.

Type	Description	Key Use Cases
AD Connector	A proxy (gateway) to your on-premise AD. Does not cache credentials.	• You want to use on-prem credentials *without* syncing them to cloud. • Use on-prem AD for authentication for AWS services (Workspaces, QuickSight).
Simple AD	Standalone Directory based on Samba 4.	• Low cost. • **Cannot** set up trust relationships with on-prem AD. • Basic user management for small cloud-only deployments.

> **Exam Tip:**
>
> - Need Trust Relationship? -> **Managed Microsoft AD**.
> - Keep users solely on-prem but auth against them? -> **AD Connector**.
> - Cheap, cloud-only, no on-prem connection needed? -> **Simple AD**.

8. AWS Control Tower

An abstraction layer on top of AWS Organizations.

- **Purpose:** The easiest way to set up and govern a secure, multi-account AWS environment. It automates the setup of a "Landing Zone."
- **Key Functionality:**
  - **Landing Zone:** Sets up a multi-account environment based on best practices (Logging account, Audit account).
  - **Guardrails:**
    - **Preventive Guardrails:** Stop non-compliant actions (implemented via **SCPs**). E.g., "Disallow creation of public S3 buckets."
    - **Detective Guardrails:** Alert on non-compliance (implemented via **AWS Config**).
  - **Account Factory:** Automates the provisioning of new accounts with pre-approved network and security configurations.

> **Exam Tip:** If the question mentions "automating the setup of a multi-account environment," "Landing Zone," or "Blueprints," choose Control Tower.